# Customer Order Automation

*Azure-Native Engineering & Implementation Specification*

For Claude / Codex | Version 1.0 | November 2026

Target ERP: Microsoft Dynamics AX 2012 • Existing integration: Boomi EDI 850 • Data: Microsoft Fabric Lakehouse

## Executive direction

Build a production-grade, single-enterprise customer purchase-order intake and automation platform. Receive orders through a Microsoft 365 mailbox; extract with Azure AI Document Intelligence and Azure AI Foundry; validate against Fabric customer/item masters; prevent duplicates across email and EDI; resolve exceptions in a modern review UI; publish approved orders to Fabric staging for an existing Boomi sales-order process that creates AX 2012 sales orders. Capture every correction for customer/template-specific learning and future local-model specialization.

## 1. Scope, assumptions and non-goals

### In scope

Mailbox monitoring; document classification and extraction; customer, ship-to, item, UOM, quantity and optional pricing validation; duplicate controls; reviewer UI; correction learning; Fabric staging; Boomi acknowledgments; AX status; admin, security, reporting and deployment.

### Out of scope for first release

No multi-tenant SaaS platform, no AP invoice processing, no direct AX connector, no new EDI translator, no autonomous model training from scratch, no uncontrolled order changes after AX creation.

### Decisions to verify

Mailbox address and Graph access; AX legal entities; Fabric table schemas and refresh cadence; existing Boomi canonical 850 structure and shared subprocess boundaries; pricing policy; whether order revisions/releases are supported; integration acknowledgment mechanism; Fabric Lakehouse vs Warehouse staging.

## 2. Target architecture and data flow

### Processing flow

Microsoft Graph shared mailbox → durable ingestion record → Blob attachment storage → document classifier → Document Intelligence OCR/layout → AI Foundry structured PO interpretation → normalized PO schema → customer/item/master validation → duplicate reservation → human review if needed → approved publication outbox → Fabric staging → Boomi canonical intake adapter → existing Boomi AX sales-order creation → ERP acknowledgment → application completion.

### Separation of responsibilities

PostgreSQL is the operational source of truth for state, corrections, jobs, audit and duplicate reservations. Fabric is the authoritative reference-data read source and order-integration staging destination. Blob holds original documents. Boomi owns AX integration. AI suggests; deterministic rules authorize staging.

### Deployment

Azure Container Apps for web/API/workers, Container Registry, Azure Database for PostgreSQL, Service Bus, Blob Storage, Key Vault, Application Insights, Document Intelligence and AI Foundry. Use managed identities, private endpoints where supported and Terraform IaC. Confirm Fabric endpoint and write method.

## 3. Identity, access and security

### Entra ID

Use OIDC for frontend and Entra-protected backend APIs. Implement app roles Administrator, Order Processor, Supervisor, Viewer and Integration Service. Enforce roles server-side and for each action.

### Graph security

Use narrowly scoped application permissions for the configured mailbox, with Exchange application access scoping where supported. Secure subscription notifications, validate authenticity and renew subscriptions; reconcile with delta queries.

### Secrets and privacy

Managed identity first; secrets in Key Vault. Encrypt data at rest/in transit. Apply document retention, PII redaction in logs, attachment malware screening, content-type validation, RBAC and auditable administrative changes. No production ERP writes during development.

## 4. Mailbox ingestion

### Requirements

Support shared or dedicated Microsoft 365 mailbox, configurable folders, Graph change notifications plus delta reconciliation, retries, rate limiting, subscription renewal, multi-attachment emails, multi-page orders and multiple orders per attachment where detectable.

### Idempotency

Persist immutable Graph identifiers plus mailbox, message and attachment identifiers, content hash and ingestion attempts. A repeated webhook, re-scan or worker retry must not create another business order.

### Operations

Email status is not workflow truth. Optionally move/categorize processed messages only after durable ingestion. Track source sender, received time, subject, attachment, classification, processing outcome and failure reason.

## 5. Document extraction and normalized schema

### Azure Document Intelligence

Use layout/OCR and evaluated prebuilt/custom extraction as appropriate. Preserve raw OCR text, page, table, tokens, polygons/bounding boxes, provider confidence, model version and original response under retention policy.

### Azure AI Foundry

Use configured deployment for schema-constrained PO interpretation, ambiguous header/line identification and exception explanation. Apply timeouts, retries, rate limits, usage/cost tracking, prompt/model versioning and output schema validation. Never invent missing IDs.

### Header schema

document_id, order_id, company, customer identity/name, customer_po_number, po_date, requested_delivery_date, bill_to, ship_to, currency, payment/shipping terms, totals, special instructions, source evidence, confidence and validation results.

### Line schema

line_id, source_line_no, customer_item_code, description, ERP_item_id, quantity, UOM, unit_price, extended_amount, requested_date, evidence, confidence, validation results. Preserve original text and normalized values separately.

### Document variants

Handle scans, searchable PDFs, page rotations, tables spanning pages, multi-order documents, missing fields, low-quality images and unknown templates. Flag ambiguous segmentation for review.

## 6. Fabric master-data access

### Customer reference

Read actual Fabric schema for ERP customer account, company, status, names/aliases, bill-to/ship-to, currency, terms and cross-references. Do not invent fields or assume all exist.

### Item reference

Read ERP item, company, item status, customer item cross-reference, descriptions, pack/UOM conversions and product identifiers. Define data freshness SLA and cache invalidation.

### Access pattern

Use supported Fabric SQL endpoint/API for reads and managed identity or approved service principal. Maintain an indexed local read model if necessary, with source timestamp and refresh health visible. Treat stale or missing master data as a controlled exception.

## 7. Matching and validation

### Customer matching

Rank exact ERP account, approved cross-reference, aliases, addresses, history and fuzzy candidates. Validate customer active and correct company, plus ship-to/bill-to rules. Show evidence and alternatives.

### Item matching

Rank exact ERP SKU, approved customer item mapping, GTIN, pack size/UOM and description similarity. Validate item active, company, quantity, conversions and configured price rules. No description-only blind auto-match.

### Validation policy

Separate errors vs warnings. Configurable required fields, dates, currency, price tolerances, quantities, totals, ship-to and approval thresholds. Revalidate after user edits and immediately before staging. AI confidence never bypasses blocking checks.

### Legal entity

Use company_id/dataAreaId in matching, duplicate keys and AX payloads. This is company-level partitioning, not a SaaS tenant model.

## 8. Duplicate prevention across email and EDI

### Business key

Initial key: normalized customer PO number + AX customer account + legal entity. Confirm handling of revisions, releases, blanket orders, cancellations and split shipments before finalizing.

### Sources to check

Application orders (including in-flight), Fabric staging, Boomi in-flight/processed EDI 850 orders and AX sales orders via authoritative indexed feed or supported query. Check both email→EDI and EDI→email collision paths.

### Atomic protection

Use PostgreSQL unique reservation/claim with transactionally enforced uniqueness for the agreed business key. The claim protects concurrent email workers; coordinate with Boomi/AX cross-channel idempotency so another channel cannot create the same order.

### Duplicate UX

Block staging, show matched existing order, source channel, ERP order and side-by-side source comparison. Permit supervisor-reviewed resolution only under explicit policy; retain audit and original claim history.

## 9. Workflow and human review

### Simple statuses

Received, Processing, Needs Review, Ready, Sent to ERP, Completed, Failed, Rejected. Map to granular internal states such as OCR_PENDING, VALIDATING, DUPLICATE_REVIEW, STAGED, BOOMI_CLAIMED and ERP_CREATED.

### Workbench

Dashboard, searchable/filterable order queue, assigned work, bulk actions only where safe, SLA/age, customer, PO, line count, confidence, error counts, status and ERP number.

### Review experience

Two-pane PDF viewer and editable order header/lines. Page navigation, zoom, text selection, bounding-box highlighting, source-to-field navigation, candidate customer/item matches, inline error messages, save draft, revalidate, approve/reject, history and keyboard-friendly editing.

### Operations UX

Failed jobs, re-drive, re-extract, document reclassification, connection health, manual escalation, audit timeline, supervisor override and accessible responsive UI.

## 10. Learning Engine and local model

### Scope

Single company application; module_code=ORDER_PROCESSING, then company/customer/document_type/template_variant/field. Keep module-scoped architecture for future modules but do not build tenant management.

### Feedback events

Record original OCR/AI values and coordinates, selected correction value and region, page, nearby anchors, document fingerprint, customer/template, prediction source/version, user, approval, final AX outcome and any reversal.

### First-correction behavior

After one correction create candidate extraction rule. On the next similar template, attempt the learned region using relative coordinates, text anchors, page structure and master-data validation. Two to three independently validated successes may qualify for trusted behavior under configurable thresholds; no guarantee from count alone.

### Customer and item learning

Maintain approved customer aliases, customer-item cross-references, UOM/pack patterns, historical outcomes, confidence, supporting evidence, conflict counts and recency. Prevent stale learned mappings from bypassing Fabric master validation.

### Rules and ML

Start with deterministic rules, historical retrieval, similarity and lightweight classifiers when justified. Collect training-quality examples (approved and AX-posted outcomes, with corrections/reversals handled) and version datasets.

### Local model roadmap

Later evaluate small open-source model hosted in Azure, optional LoRA/QLoRA fine-tuning on approved company order data, shadow mode, evaluation, registry, controlled promotion, rollback and quantized inference. Do not train an LLM from scratch or fine-tune on unverified corrections.

### Controls

Explain every suggestion with evidence; configurable auto-fill vs auto-stage thresholds; monitor accuracy/drift; model output always revalidated; maintain audit and no cross-module contamination.

## 11. Fabric staging and Boomi reuse

### Canonical reuse

Inspect existing Boomi inbound EDI 850 maps and reusable subprocesses. Build a thin Fabric-to-canonical adapter; reuse customer/item transformations, AX order creation, retries and error handling without disrupting live EDI.

### Staging logical schema

Header: staging_order_id, source_channel=EMAIL_PO, company, ERP_customer, customer_PO, dates, ship-to, currency, totals, line_count, validation_state, publish_state, correlation_id, schema_version. Lines: staging_order_id, line_no, ERP_item, customer_item, qty, UOM, price, dates. Include acknowledgments/errors.

### Fabric technical constraint

Lakehouse SQL analytics endpoint is generally read-oriented; do not assume transactional DML. Select a supported Delta write path (pipeline/notebook/API) or evaluate Fabric Warehouse if Boomi needs SQL-based operational staging and acknowledgments. Validate against actual Fabric capabilities and Boomi connector behavior.

### Publish safety

Use durable transactional outbox in operational DB, retryable publisher and reconciliation. Only expose complete validated header+lines; reconcile line count/checksum. Stable IDs and idempotent writes avoid duplicate staging.

### Boomi acknowledgment

Agree on claim/read semantics and callback/status ingestion. Track Boomi accepted, AX created, AX sales order ID, error codes, timestamps and retries. A staged record is not a completed order.

### EDI coexistence

Regression-test existing EDI 850 processing. Shared duplicate controls must account for orders arriving through either channel; preserve canonical EDI functionality.

## 12. Operational data model

### Core entities

mail_message, mail_attachment, source_document, extracted_order, order_header, order_line, order_validation, order_status_history, duplicate_claim, processing_job, job_attempt, outbox_event, integration_ack, audit_event.

### Learning entities

learning_event, template_variant, extraction_pattern, pattern_evidence, customer_alias, customer_item_mapping, learning_rule, model_registry, training_dataset, prediction_evaluation.

### Keys and consistency

Use UUID internal IDs; explicit company_id, source_channel, immutable document hashes, unique external identifiers, row version/concurrency fields, created/updated timestamps and retention controls. Index customer/PO/company and queue/status queries.

### Migrations

Versioned schema migrations, rollback procedures, seed roles/config only, no embedded secrets. Preserve original document evidence and immutable decision history.

## 13. API and service contracts

### Core APIs

GET /orders; GET /orders/{id}; PATCH /orders/{id}; POST /orders/{id}/revalidate; POST /orders/{id}/approve; POST /orders/{id}/reject; GET /orders/{id}/history; GET /orders/{id}/document; GET /orders/{id}/matches.

### Learning APIs

POST /learning/feedback; GET /learning/patterns; GET /learning/explanations/{order_id}; GET /learning/metrics; POST /learning/patterns/{id}/approve; POST /learning/patterns/{id}/disable.

### Admin/integration APIs

GET/PUT /settings; POST /connections/{type}/test; GET /jobs; POST /jobs/{id}/retry; POST /integration/acknowledgments (Entra service auth); GET /health; GET /metrics. Adapt routes to repository conventions.

### Contract requirements

OpenAPI, strict JSON validation, pagination, authorization, correlation IDs, idempotency keys, structured errors, rate limits and optimistic concurrency for reviewer edits.

## 14. UI page inventory

### Pages

Sign-in; dashboard; order inbox; order detail/review; duplicate comparison; exceptions; completed orders; integration monitor; failed jobs; learning rules/templates; learning analytics; customer aliases; customer item mappings; users/roles; settings/connections; audit log.

### UX acceptance

Every order status has clear next action; one-screen review for routine exceptions; document evidence visible beside editable fields; warnings distinct from errors; reviewer actions saved without losing progress; no inaccessible color-only status signals.

## 15. Infrastructure, CI/CD and operations

### Azure resources

Separate dev/test/prod resource groups and identity boundaries; Container Apps web/API/workers, PostgreSQL, Service Bus, Blob, Key Vault, ACR, Monitor/App Insights, Document Intelligence, AI Foundry and networking. Terraform modules and parameterized environments.

### Pipelines

Build, lint, unit tests, security scan, container image, database migration check, integration tests and controlled deploy. Infrastructure changes reviewed. Deploy rollback and configuration backups documented.

### Reliability

Dead-letter queues, retry/backoff, poison-message quarantine, stuck job sweeper, Graph delta reconciliation, Fabric publication reconciliation, Boomi acknowledgment timeout alert, backup/restore and disaster recovery tests.

### Observability

Structured logs, trace/correlation IDs across Graph→OCR→AI→Fabric→Boomi→AX, dashboards for latency, queue depth, error rates, costs, STP, matching quality and model drift. Never log raw secrets.

## 16. Test matrix and acceptance criteria

### Critical test cases

Clean single PO; multiple attachments; multiple POs; scanned/rotated PDF; multi-page line tables; unknown customer; wrong customer corrected by PDF highlight; unknown SKU corrected; UOM mismatch; duplicate against email; duplicate against EDI/AX; concurrent duplicate arrival; revised PO; master-data stale; Graph throttling; OCR timeout; AI malformed JSON; Fabric unavailable; partial publish; Boomi retry; AX timeout; service restart; unauthorized override; learned template reused.

### Acceptance gate

No invalid/duplicate order reaches AX; all orders traceable end-to-end; existing EDI regression passes; review UX supports field highlighting and correction; at least one learned template reuses a correction on a later PO; failures recover without duplicate side effects; RBAC and audit verified.

### Nonfunctional targets

Agree measurable SLOs for availability, mailbox-to-review latency, normal order processing time, concurrency, maximum document size, retention, RPO/RTO and extraction/matching accuracy before production sign-off.

## 17. Phased implementation backlog

### Phase 0 – Discovery

Obtain Boomi EDI 850 canonical payload and process map, AX duplicate-check approach, Fabric schemas, mailbox permissions, PO samples, legal entities, business validation policy and hosting/security standards. Produce ADRs and gap analysis.

### Phase 1 – Foundation

Repo, CI/CD, Terraform, Entra auth, roles, PostgreSQL migrations, queues, Blob, secrets, app shell, audit and health endpoints.

### Phase 2 – Intake and extraction

Graph ingestion, attachments, document storage, Document Intelligence, AI Foundry structured schema, evidence coordinates, status machine and exception handling.

### Phase 3 – Master matching and duplicate

Fabric read model, customer/item matching, UOM validation, duplicate claims, cross-channel ERP/EDI duplicate lookup, rules and tests.

### Phase 4 – Review UX

Queue, PDF split viewer, editing, field highlighting, validation, approval, duplicate review, activity timeline and role enforcement.

### Phase 5 – Boomi integration

Choose Fabric staging target, outbox publisher, canonical adapter, Boomi consumption and AX acknowledgments, reconciliation and EDI regression.

### Phase 6 – Learning

Learning event store, first-correction layout reuse, customer/item aliases, pattern administration, explainability and learning dashboard.

### Phase 7 – Hardening

Performance, security, DR, monitoring, UAT, operational documentation, pilot, production rollout and hypercare.

### Phase 8 – Optional specialized model

Dataset generator, lightweight ML evaluation, Azure-hosted local inference, shadow mode, fine-tuning assessment, model registry and controlled promotion.

## 18. Claude/Codex execution instructions

### First response required

Do not begin coding immediately. Inspect repo and available artifacts; identify reusable patterns; return architecture diagram, proposed repo tree, ERD, normalized PO JSON schema, API list, Azure resource list, Fabric/ Boom i contract questions, threat model, implementation backlog, risks and assumptions.

### Development rules

Implement vertical slices with tests and migrations. No placeholder production integrations, hard-coded secrets, unreviewed infrastructure deployment, production mailbox changes or ERP writes. Use feature flags and mocks only for test environments; clearly label unresolved external dependencies.

### Per-phase deliverables

Code, tests, updated OpenAPI, migrations, Terraform/config changes, runbook, sample payloads, evidence of passing tests, deployment instructions and known limitations.

### Definition of done

Validated orders flow mailbox→review/auto-validation→Fabric→Boomi→AX with feedback; duplicates blocked across channels; corrections learned; operations visible; security and tests pass; EDI remains unaffected.

## Appendix A – Recommended handoff prompt

Claude/Codex: Treat this document as the authoritative product specification. Review the existing AP Automation patterns only as design inspiration and the existing Boomi EDI 850 implementation as the AX integration baseline. First inspect repository and integration artifacts, produce an implementation gap analysis and architecture decision log, identify missing company-specific

information, and propose a sequenced backlog. Do not alter production infrastructure, Boomi processes or AX data without approval. Then implement one tested phase at a time.